IMPORTANT DIFFERENCE between LOG files from [mcr_2405] –vs– [2603] versions of DMON firmware/d3 programs:

- **[mcr_2405]** uses **milliseconds** represented by **4-character** field
- [2603] uses microsecond 8-character field for $DUTY lines; **1 millisecond = 1000 microseconds**

UVI2_AroundSTX_20250306 mission used **[mcr_2405]** version DMON update release
*(firmware **LF2_HF240d1_5m.bin**, d3host.exe **version 2.2**, d3read.exe **version 2.0**, and reformat_dmon_wav_files.sav **version 1**)*

The data payload and checksum of $DUTY lines in DMON2 log files can be decoded as follows…

Example $DUTY line in log file (from UVI2_AroundSTX_20250306_050.log):

1742680666,1594,$DUTY,67DF244701F967DF3257000520000018300 6DDD00*07

Characters 2-9 (n=8) after $DUTY: Start date-time of recording session in Unix epoch UTC

67DF2447 hexadecimal = **1742677063** decimal (sec Unix epoch) = **2025-03-22 20:57:43** UTC

Characters 10-13 (n=4): Milliseconds after start date-time in Unix epoch UTC

01F9 hex = **505** decimal (milliseconds) = **0.505** seconds →

2025-03-22 20:57:43 UTC + 505 milliseconds = **2025-03-22 20:57:43.505** UTC

Characters 14-21 (n=8): End date-time of recording session in Unix epoch UTC

67DF3257 hex = **1742680663** decimal (sec Unix epoch) = **2025-03-22 21:57:43** UTC

Characters 22-25 (n=4): Milliseconds after end dt in Unix epoch UTC

0005 hex = **5** decimal (milliseconds) = **0.005** seconds →

2025-03-22 21:57:43 UTC + 5 milliseconds = **2025-03-22 21:57:43.005** UTC

Character 26 (n=1): Hydrophone numeric ID **2 = LF hydrophone, 3 = HF hydrophone, 1 = MF hydrophone*

2 = LF hydrophone

Characters 27-34 (n=8): Sequential recording session number for *specific hydrophone (LF/HF)* during mission

00000183 hex = **387** decimal = **387th** recording session for LF hydrophone during mission

Characters 35-42 (n=8): Number of samples in current recording session

006DDD00 hex = **7200000** decimal = **7,200,000** total samples in the 387th LF recording session

Characters 44-45 (n=2): Checksum indicating if any errors exist in the line

07 **see last page of this tab for details on calculating the checksum for each line*

```
OPTIONAL BONUS: Check if n samples matches expected value calculated from duration and sample rate
  –  Duration = 2025-03-22 20:57:43.505 UTC to 2025-03-22 21:57:43.005 UTC = 3,599.500 sec
  –  Sample rate = 2 kHz = 2,000 Hz = 2,000 samples/sec
  –  Expected total samples = 3,599.500 sec × 2,000 samples/sec = 7,199,000 samples
= OFF by 1,000 samples (aka 0.5 seconds) from expected
```

The data payload and checksum of $START lines in DMON2 log files can be decoded as follows…

Example $START line in log file (from UVI2_AroundSTX_20250306_001.log):

1741287466,8468,$START,67C9F02A00004C232435 00010000*65

Characters 2-9 (n=8) after $START: Date-time in Unix epoch UTC

67C9F02A hexadecimal = **1741287466** decimal (sec Unix epoch) = **2025-03-06 18:57:46** UTC

Characters 10-13 (n=4): Milliseconds after date-time in Unix epoch UTC

0000 hex = **0** decimal (milliseconds) →

2025-03-06 18:57:46 UTC + 0 milliseconds = **2025-03-06 18:57:46.000** UTC

Characters 14-21 (n=8): DMON2 board ID for the specific DMON2 used in the mission

4C232435

Characters 22-29 (n=8): Flash memory size (MB)

00010000 hex = **65536** decimal (MB) = **64** GB (binary units) of flash memory available on the DMON2

**from Baumgartner "The DMON2 application firmware saves data (e.g., audio, logs, configuration information) to flash memory in chunks as they are generated in real time, and d3host assembles those chunks into .dtg files that are then unpacked into .wav, .log, .xml and .err files by the d3read program"*

Characters 31-32 (n=2): Checksum indicating if any errors exist in the $START line

65 **see last page of this tab for details on calculating the checksum for each line*

The data payload and checksum of $END lines in DMON2 log files can be decoded as follows…

Example $END line in log file (from UVI2_AroundSTX_20250306_058.log):

1742919208,7927,$END,67E2D62800084C2324351*22

Characters 2-9 (n=8) after $END: Date-time in Unix epoch UTC

67E2D628 hexadecimal = **1742919208** decimal (sec Unix epoch) = **2025-03-25 16:13:28** UTC

Characters 10-13 (n=4): Milliseconds after date-time in Unix epoch UTC

0008 hex = **8** decimal (milliseconds) = **0.008** seconds →

2025-03-25 16:13:28 UTC + 8 milliseconds (0.008 sec) = **2025-03-25 16:13:28.008** UTC

Characters 14-21 (n=8): DMON2 board ID for the specific DMON2 used in the mission

4C232435

Character 22 (n=1): Shutdown code indicating DMON2 is properly shutting down and entering hibernation mode

1

Characters 24-25 (n=2): Checksum indicating if any errors exist in the $END line

22 **see last page of this tab for details on calculating the checksum for each line*

How to calculate expected checksum values of DMON2 log file lines

- All characters between "$" and "*" in line get checked
- Calculate binary "running result" value by stacking binary values of last row and current row
  - **different bits between rows = 1 and same bits = 0**

0 1 0 1 0 0 1 1 ← binary running result from all previous characters (row #1, col 5 below)
0 1 0 1 0 1 0 0 ← binary of current character derived from ASCII decimal (row #2, col 4 below)
0 0 0 0 0 1 1 1 ← output binary running result for current row (row #2, col 5 below)

Example checksum calculation for line:
`1741287466,8468,$START,67C9F02A00004C23243500010000*65`

#	Char	ASCII decimal	Binary	Binary running result	= Decimal	= Hexadecimal result final row
	start			*00000000*	*0*	
1	S	83	01010011	01010011	83	
2	T	84	01010100	00000111	7	
3	A	65	01000001	01000110	70	
4	R	82	01010010	00010100	20	
5	T	84	01010100	01000000	64	
6	,	44	00101100	01101100	108	
7	6	54	00110110	01011010	90	
8	7	55	00110111	01101101	109	
9	C	67	01000011	00101110	46	
10	9	57	00111001	00010111	23	
11	F	70	01000110	01010001	81	
12	0	48	00110000	01100001	97	
13	2	50	00110010	01010011	83	
14	A	65	01000001	00010010	18	
15	0	48	00110000	00100010	34	
16	0	48	00110000	00010010	18	
17	0	48	00110000	00100010	34	
18	0	48	00110000	00010010	18	
19	4	52	00110100	00100110	38	
20	C	67	01000011	01100101	101	
21	2	50	00110010	01010111	87	
22	3	51	00110011	01100100	100	
23	2	50	00110010	01010110	86	
24	4	52	00110100	01100010	98	
25	3	51	00110011	01010001	81	
26	5	53	00110101	01100100	100	
27	0	48	00110000	01010100	84	
28	0	48	00110000	01100100	100	
29	0	48	00110000	01010100	84	
30	1	49	00110001	01100101	101	
31	0	48	00110000	01010101	85	
32	0	48	00110000	01100101	101	
33	0	48	00110000	01010101	85	
34	0	48	00110000	01100101	101	**65 (checksum for this $START line!)**